

前端开发实习生

上海哔哩哔哩科技有限公司 用户技术中心 - 生态技术部 - 前端技术部 - C端&基建组 2025.10 - 2026.02

工作概述

负责 Dejavu Web APM（应用性能监控）平台的前端开发与优化工作，涵盖错误监控、性能分析、数据可视化等核心功能模块。独立完成多个复杂业务需求的全栈开发，并主导平台UI升级改造，显著提升了用户体验和系统性能。

技术栈: Vue 3 + TypeScript + Vite + Pinia + Naive UI + ECharts + Monaco Editor

一、APM 错误告警分析系统【核心项目】

项目背景

Dejavu APM平台需要支持多种错误类型的实时监控与分析，为开发者提供快速定位和修复问题的能力。

主要职责

1. 错误监控架构设计与实现

- 独立完成 5 种错误类型（JS运行时错误、Promise拒绝、API调用失败、资源加载失败、白屏检测）的告警分析页面开发
- 基于 Pinia Store 按错误类型分离设计，实现 useAlarmJsStore、useAlarmApiStore、useAlarmWhiteStore 等专项 Store，提升状态管理的可维护性
- 梳理 alarm-analysis 工作流程，设计通用组件与个性化组件的分离架构

2. 错误详情表格重构

- 主导 error-data-table 组件的全面重构，统一 5 种错误类型的表格展示逻辑
- 实现错误详情的多维度聚合展示（按错误类型、时间、用户影响范围等），支持嵌套数据的展开与收起
- 修复 resource 模块 message 字段错误、白屏检测下 resource 相关错误板块的数据渲染问题

3. AI 修复建议集成

- 集成 AI 流式输出功能，为错误提供智能修复建议（getAIFixSuggestions()）
- 实现源代码映射功能（getErrorSourceFile()），支持错误定位到具体代码行

4. 性能优化

- 针对告警分析页面，优化 API 调用策略：通过 Store 统一管理公共数据，消除组件间重复请求
- 实现 Iceberg 查询模式（大数据集采样），解决海量错误数据的查询性能问题

技术亮点

- 掌握 OneService API ID 驱动模式（api_2056 白屏、api_4264 JS 错误等）
- 理解 多平台 Token 架构（_http()、_http2()、_httpAlarm() 三平台模式）
- 实现响应式图表（ECharts），取消容器大小调整时的默认重新加载数据逻辑

二、APM 平台 UI 升级改造【体验优化】

项目背景

原有 APM 平台首页和 Market 页面存在信息架构不清晰、视觉风格陈旧等问题，需要进行全面的 UI/UX 升级。

主要职责

1. 首页 (Index) 全面改版

- 参与首页设计稿的产出与评审，完成新版首页的完整开发
- 实现 项目健康度排序功能，支持按变化情况（增长率）排序，并将健康分 80 分以上/以下的项目分离展示
- 新增 紧急优化项显示，帮助用户快速识别需要重点关注的项目

2. Market 页面改造

- 完成 Market 页面 Tab 切换部分的 UI 调整
- 优化 项目大观表格：实现懒加载机制，减少首次打开时的请求次数，提升页面加载速度
- 统一配色方案，优化数据可视化效果

3. 移动端适配

- 完成 APM 周报功能的 H5 响应式布局，支持移动端访问

4. 通用组件迁移

- 将 alarm-user-impact、agg-drawer 等通用组件进行重构与迁移，提升代码复用率
- 统一 agg-content 新旧版本，简化组件调用逻辑

上线成果

- 成功上线 Index & Market 部分，用户反馈体验显著提升
- 修复线上 localStorage key 值重复导致的数据污染问题

三、APM 周报订阅系统【全栈开发】

项目背景

为了让用户定期获取项目性能与错误监控数据，需要开发一套完整的周报订阅与推送系统。

主要职责

1. 前端功能开发

- 独立完成 周报首页 与 周报详情页 的设计与开发
- 实现统计数据渲染、变化率数据展示、健康分计算规则弹窗（改为链接跳转）
- 开发 项目订阅功能：查询订阅状态、订阅/取消订阅操作，解决请求字段对齐问题

2. 后端接口联调

- 参与后端 API 设计，完成接口数据类型更改与字段调整
- 解决 log_date 时间字段的统一处理方案
- 调通订阅、取消订阅、查询已订阅项目等核心接口

3. 数据处理优化

- 实现 自定义指标的本地存储（localStorage），支持跨项目数据持久化
- 简化 statTrends 处理逻辑，优化首页日期获取逻辑
- 修复自定义指标跨项目本地存储导致的数据污染问题

4. 数据可视化

- 使用 ECharts 实现周报数据的多维度图表展示（时间趋势、错误占比等）
- 实现从首页到详情页的跳转，支持健康分弹窗展示

上线成果

- 成功上线周报订阅功能，支持用户定期获取项目性能报告

四、APM 性能大盘分析【进行中】

项目背景

平台需要提供更细粒度的性能分析能力，帮助开发者识别性能瓶颈。

主要职责

1. 性能看板开发

- 完成 性能看板与业务分类下的性能分析 功能
- 实现 终端性能对比，支持跨终端（PC、H5、App内/App外）的性能数据对比
- 针对 H5 页面，细分查询 App 内 / App 外的性能表现

2. 筛选条件优化

- 增加筛选条件查询项目性能列表，提供筛选指引，默认查询当前项目上报最多的类型
- 优化指标数据修正逻辑：针对昨日/近7天/近30天，分别与前天/前周/前月对比

3. 数据兜底处理

- 完成 dejavu-apm-site 的数据兜底为 0 的处理逻辑，提升页面稳定性

技术亮点

- 掌握 Lighthouse 性能指标（FCP、LCP、TTI 等）的前端采集与分析
- 理解性能监控的多维度聚合查询（OneService 数据表聚合）

五、贝壳支付业务需求【业务支撑】

项目背景

支持贝壳支付业务的两个关键功能：税务申报异常处理与服务商多协议签署。

主要职责

1. 税务申报异常页面

- 完成静态页面开发，包括异常提示、引导流程等
- 后端接口联调，完成动态数据传入与弹窗部分开发
- UI 调整并提测，通过测试

2. 服务商多协议签署

- 完成切图与页面开发，实现多协议列表展示与签署流程
- 接入 list 接口与 contract 接口，处理人脸识别与验证字段的逻辑
- 查询可用接口入参调整，优化用户签署体验

技术学习

- 学习 jsBridge 跨端通信，理解 H5 与 Native 的交互机制
- 掌握空值合并运算符 (??) 的使用，避免将 0 判为假值

六、工程化实践与技术学习

1. Git 工作流规范

- 掌握 Dejavu 发布流水线：权限申请 → MR 提交 → Code Review → 发布上线
- 学习设置跟踪上游分支，规范分支管理

2. 代码质量优化

- 修复 jinkela-h5 (teenagers) 与 jinkela-account-h5 (security) 的 bug
- 修复账号 PC 零散页面合集的代码异味

3. 技术学习

- 深入学习 Vue Router 的不同历史记录模式与滚动设置
- 理解 H5 新增标签与 API
- 学习 Mixin 设计模式

工作亮点总结

- ✅ **独立负责核心模块**：从需求分析、技术设计到开发上线，全流程参与
- ✅ **性能优化能力**：通过请求优化、懒加载、数据兜底等手段提升系统性能
- ✅ **全栈开发经验**：参与后端接口设计与联调，具备完整的前后端协作能力
- ✅ **工程化思维**：重视代码复用、组件化设计、状态管理优化
- ✅ **快速学习能力**：在短时间内掌握 APM 监控领域的业务逻辑与技术栈